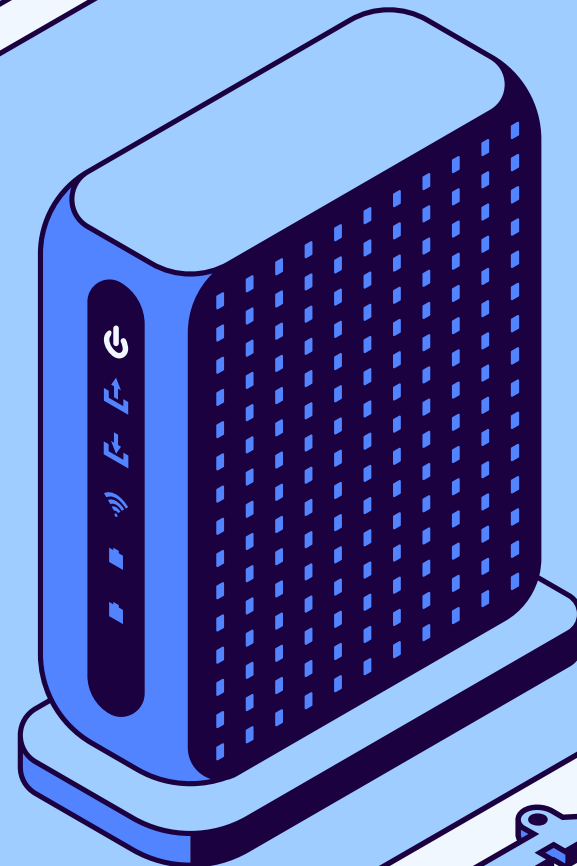
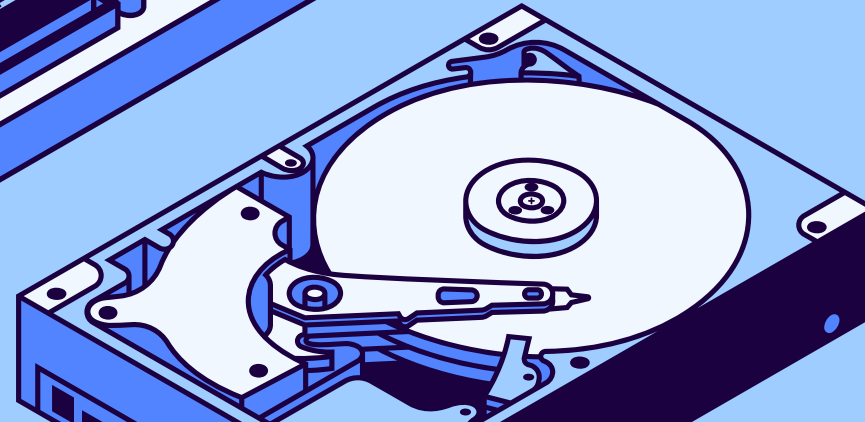
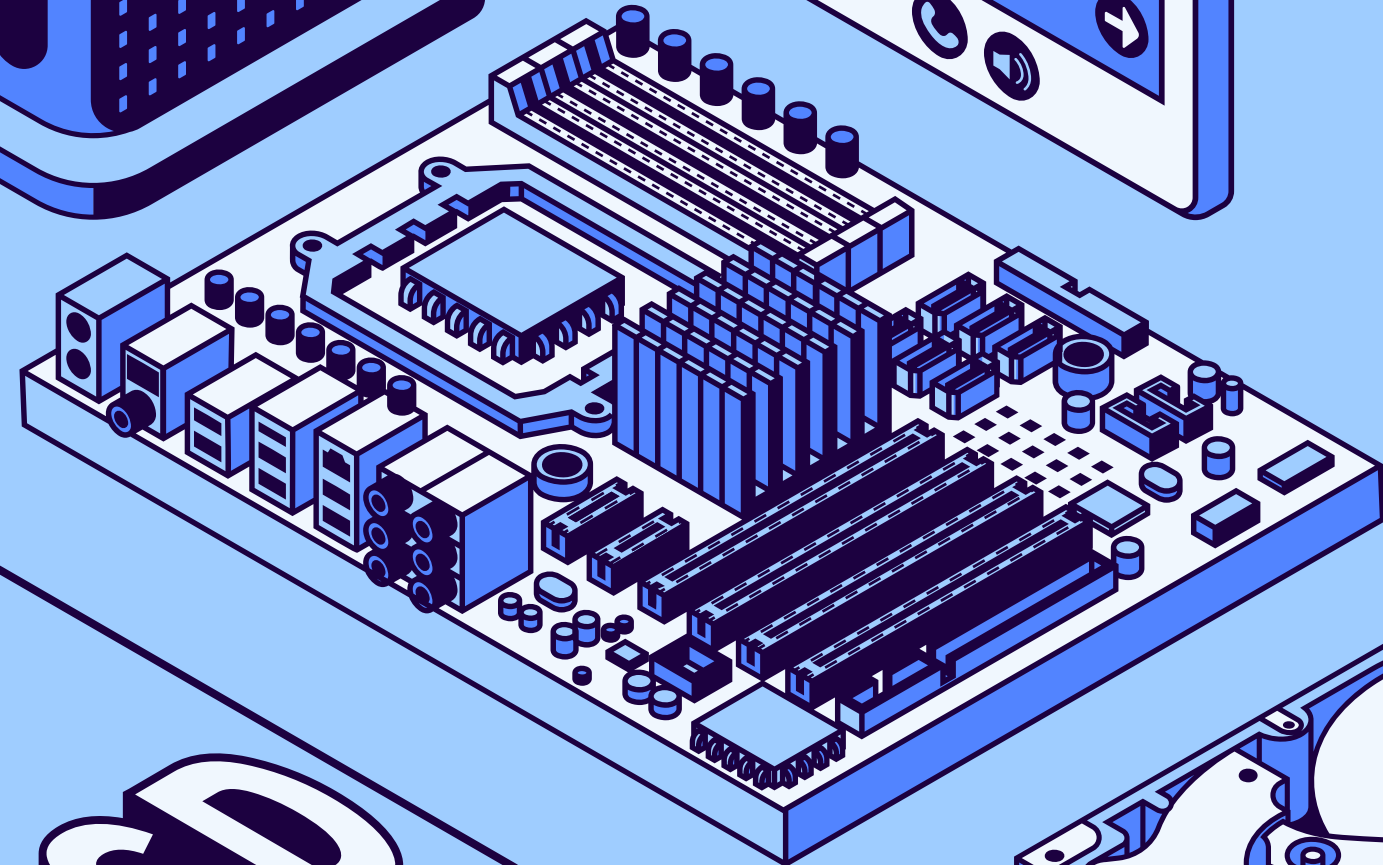




EOS - FINAL

視覺暫留 (POV) 互動遊戲顯示器

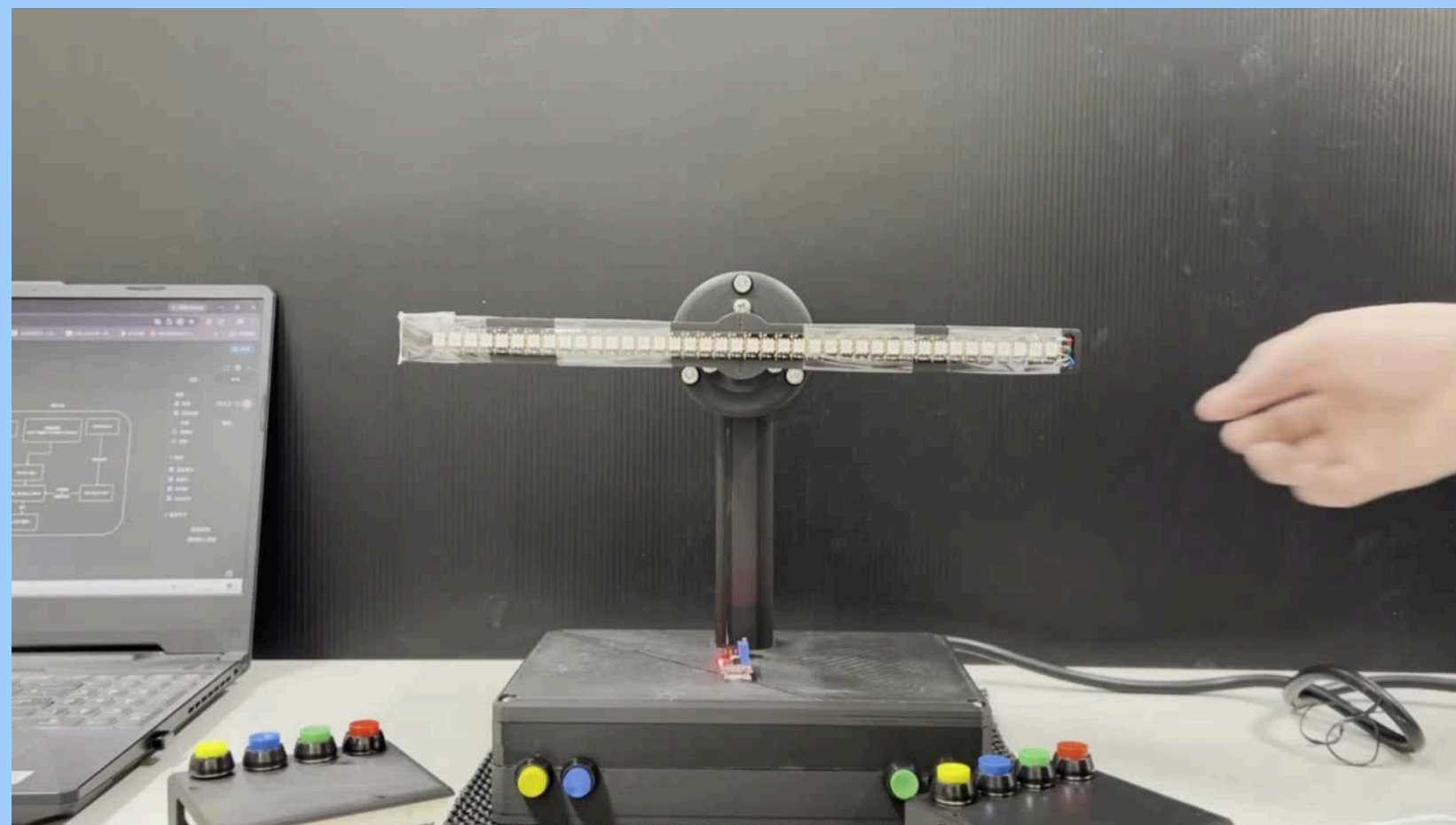
第二組



314511041 張彤翰
314512063 林汶憲
a142108 黃宇成

專題說明 - 視覺暫留 (POV) 互動遊戲顯示器

利用「視覺暫留（Persistence of Vision）」結合高速旋轉的 LED 發光元件，精準控制 LED 於正確角度亮起打造出一個 360 度的環狀立體顯示螢幕。



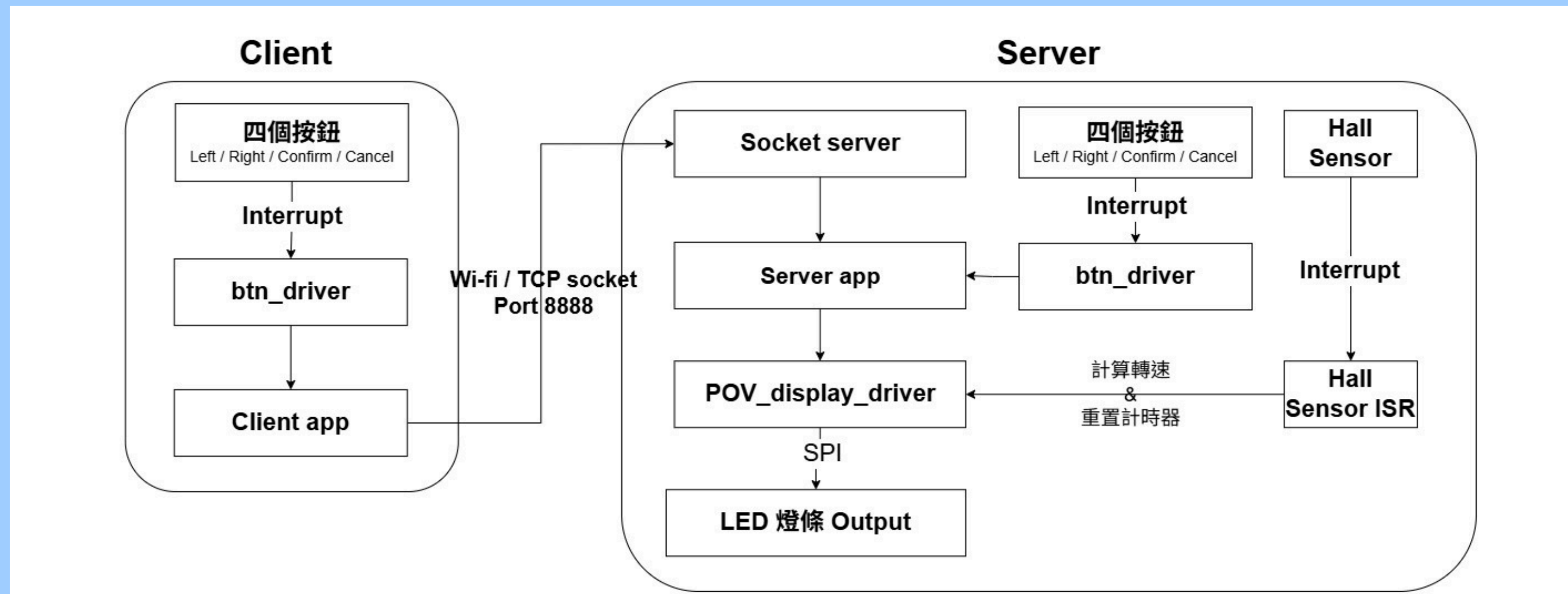
系統連線架構 (TCP client-server)

Server 端(主機)

- 啟動 TCP server 監聽 port 8888
- 每個 client 連線後分配 player id
- 每個 client 使用獨立 thread 接收按鍵
- 按鍵資料 push 到 shared key queue

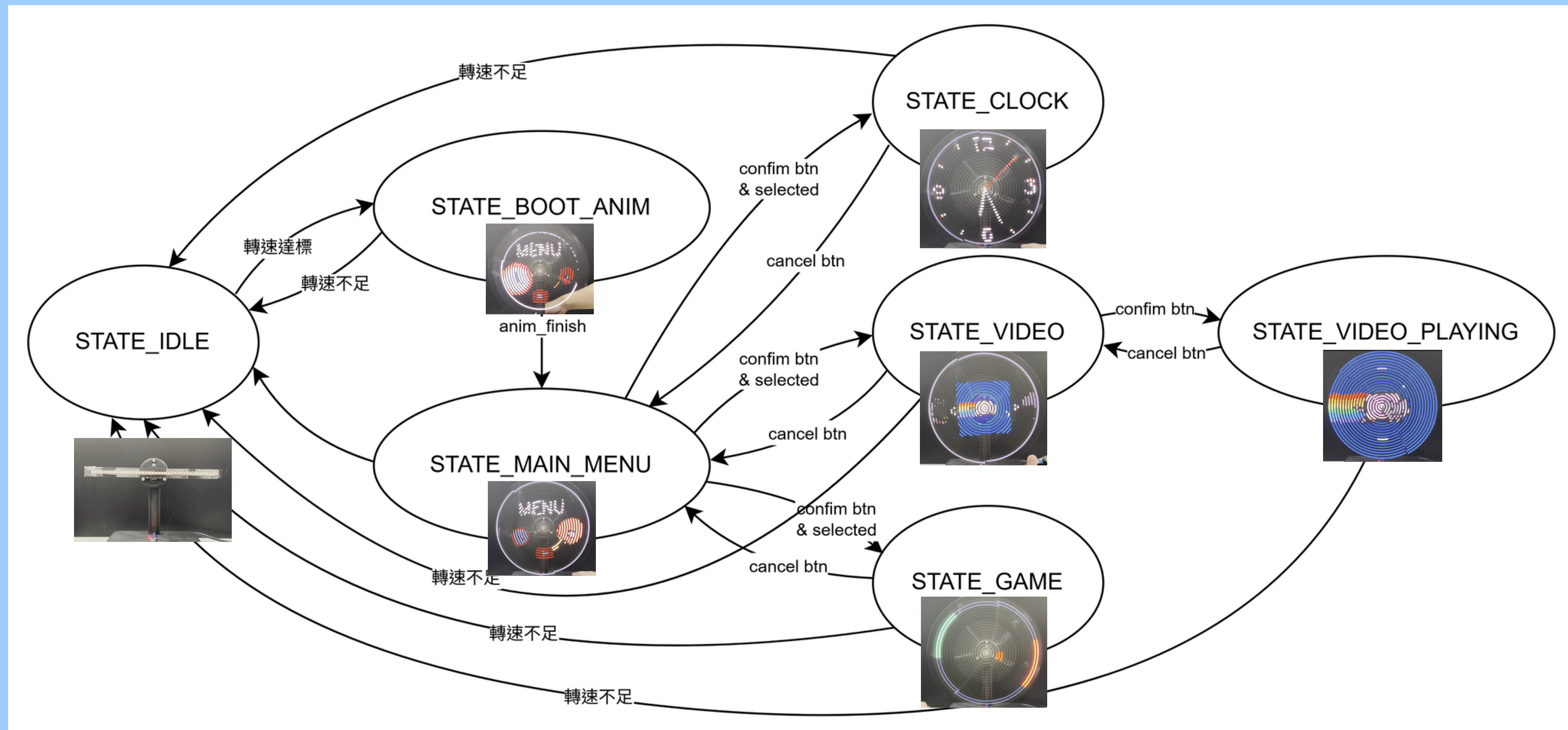
Client 端(手把)

- 安裝 driver 並啟用 device
- 使用 SIGIO 接收按鍵事件
- 將按鍵透過 TCP socket 傳給 server



Server FSM 架構

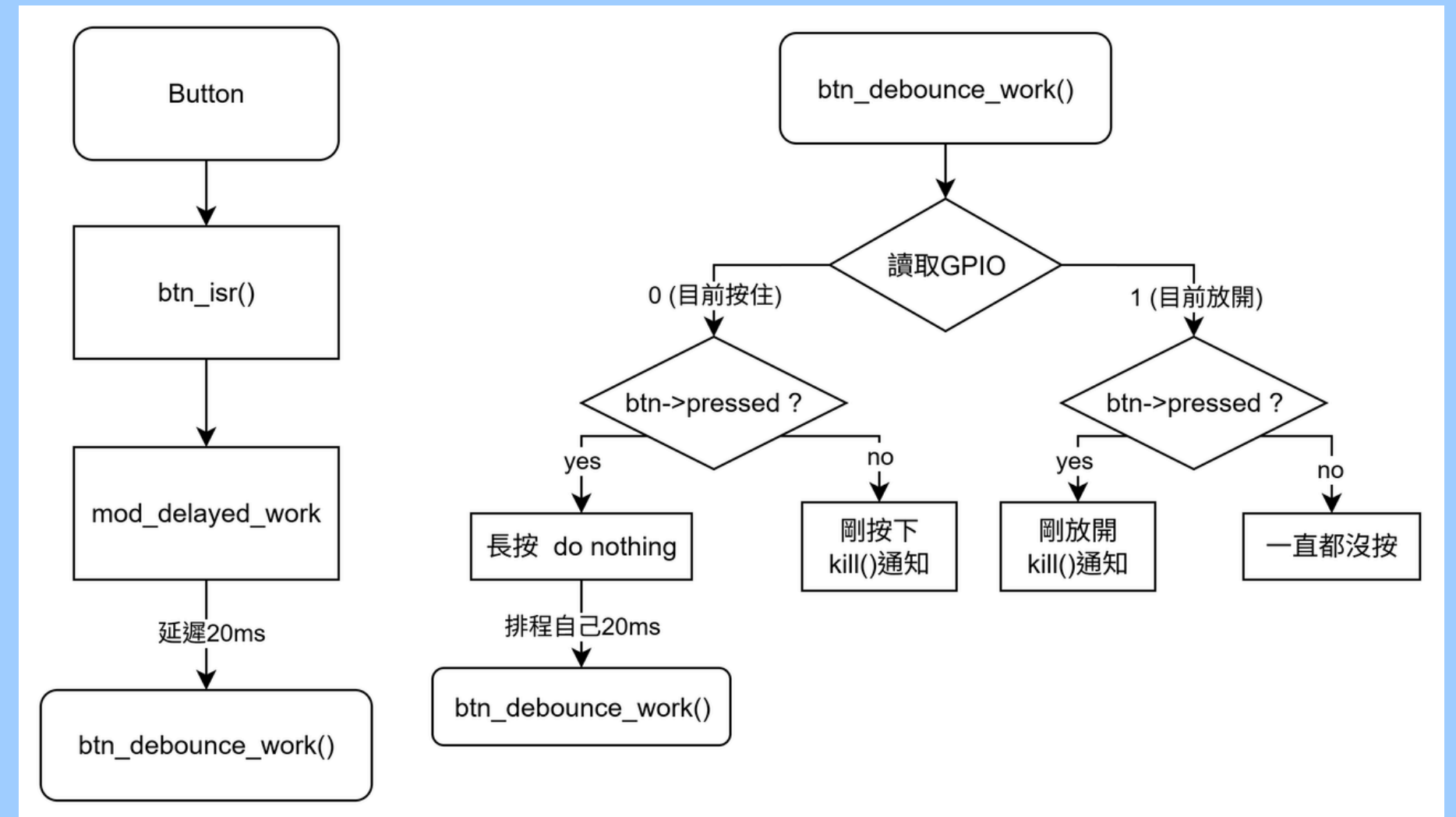
主程式使用狀態機 FSM，管理整個系統流程



Kernel Space

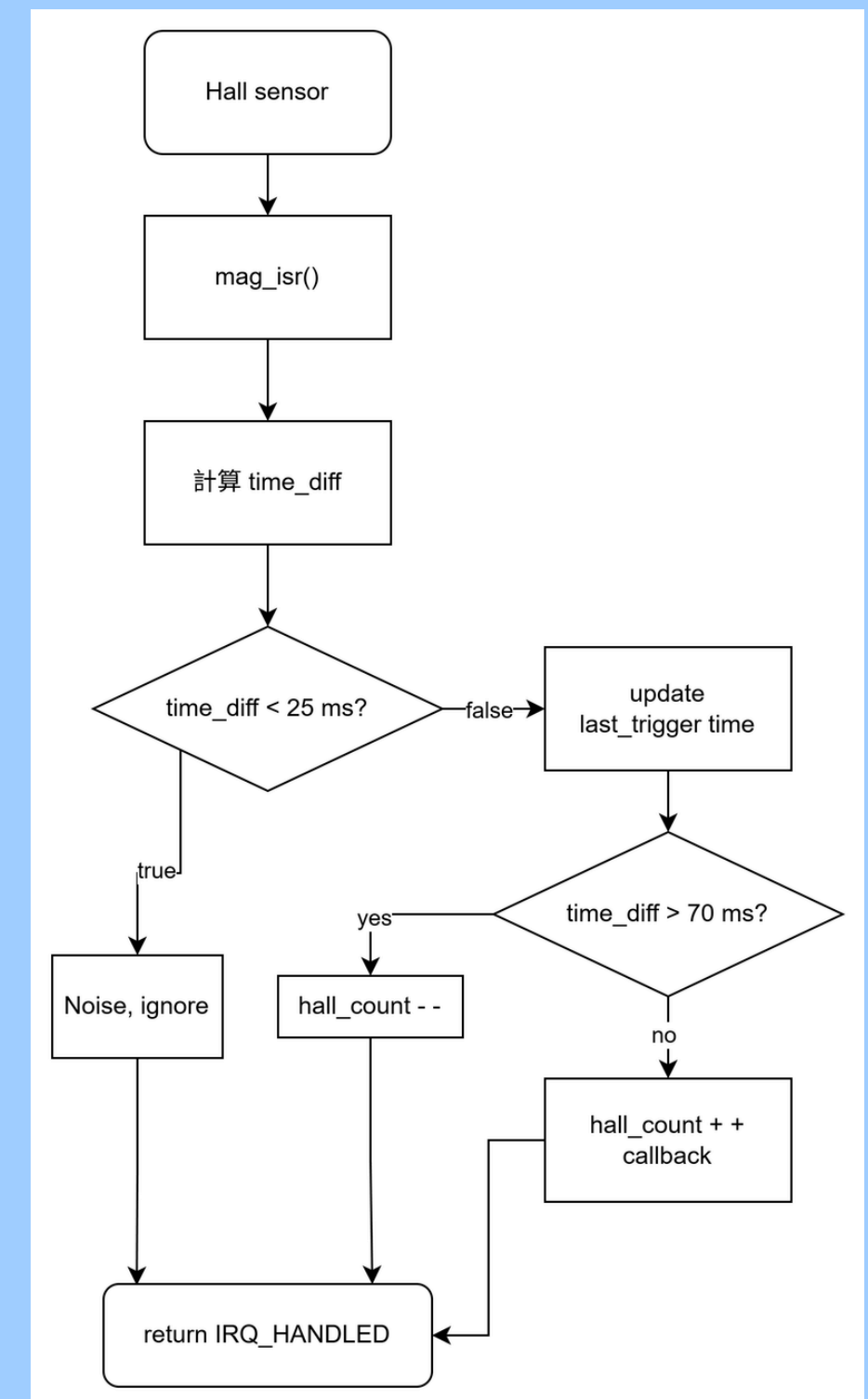
Button Driver : 遙控器按鈕

- 四顆按鍵：left、right、confirm、cancel
- ISR 偵測按下與放開，signal通知userspace讀取狀態
- Worker task 實作 debounce 與與狀態判斷
- 即使放開時 rising interrupt 沒有觸發，delayed work 也會在下一次檢查時發現已經 release。

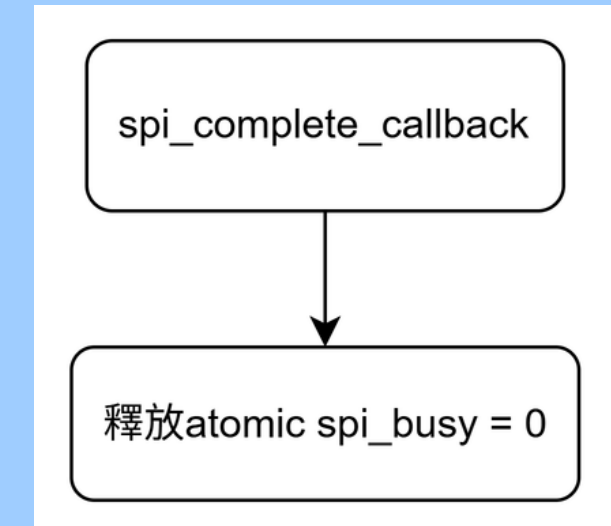
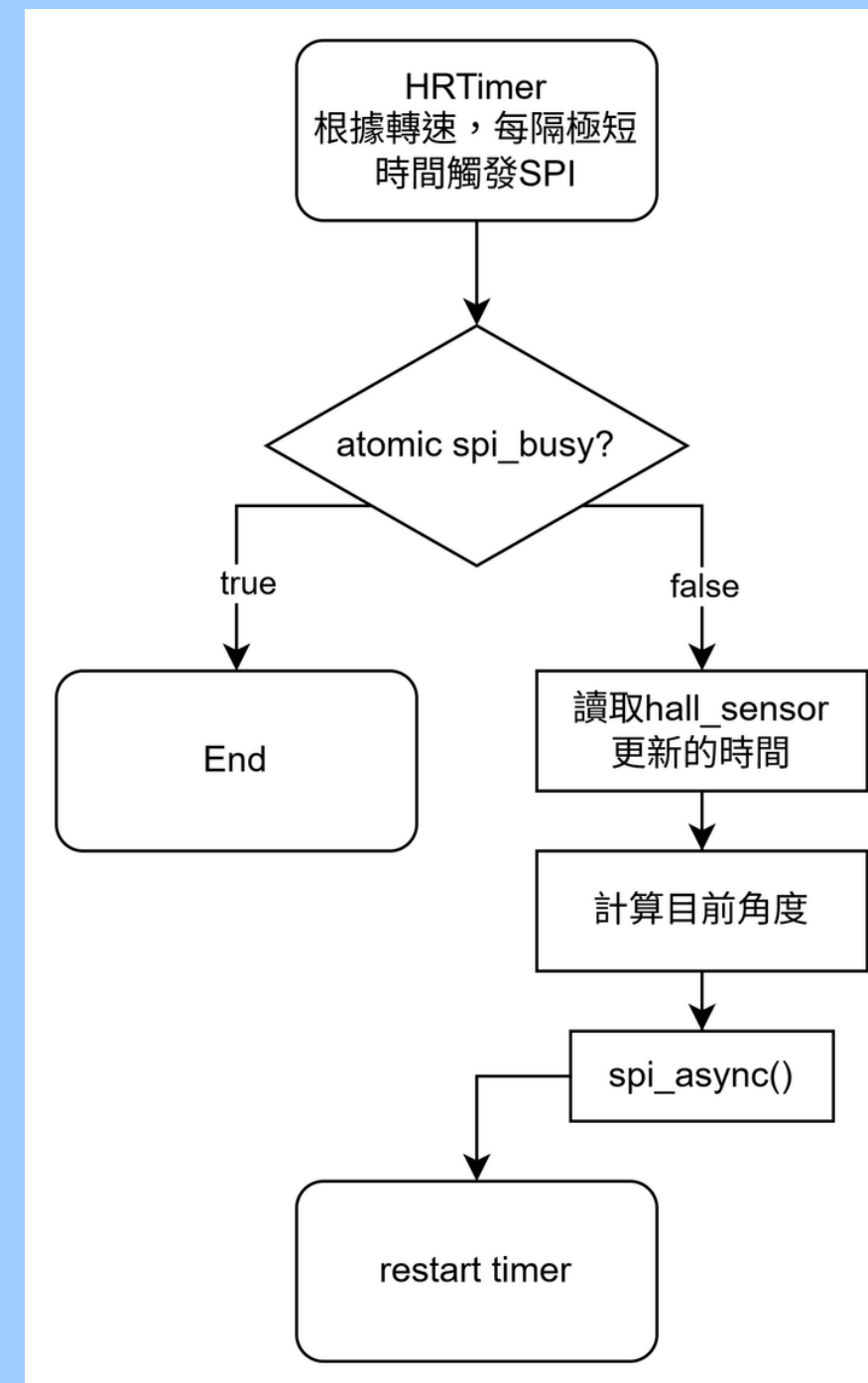
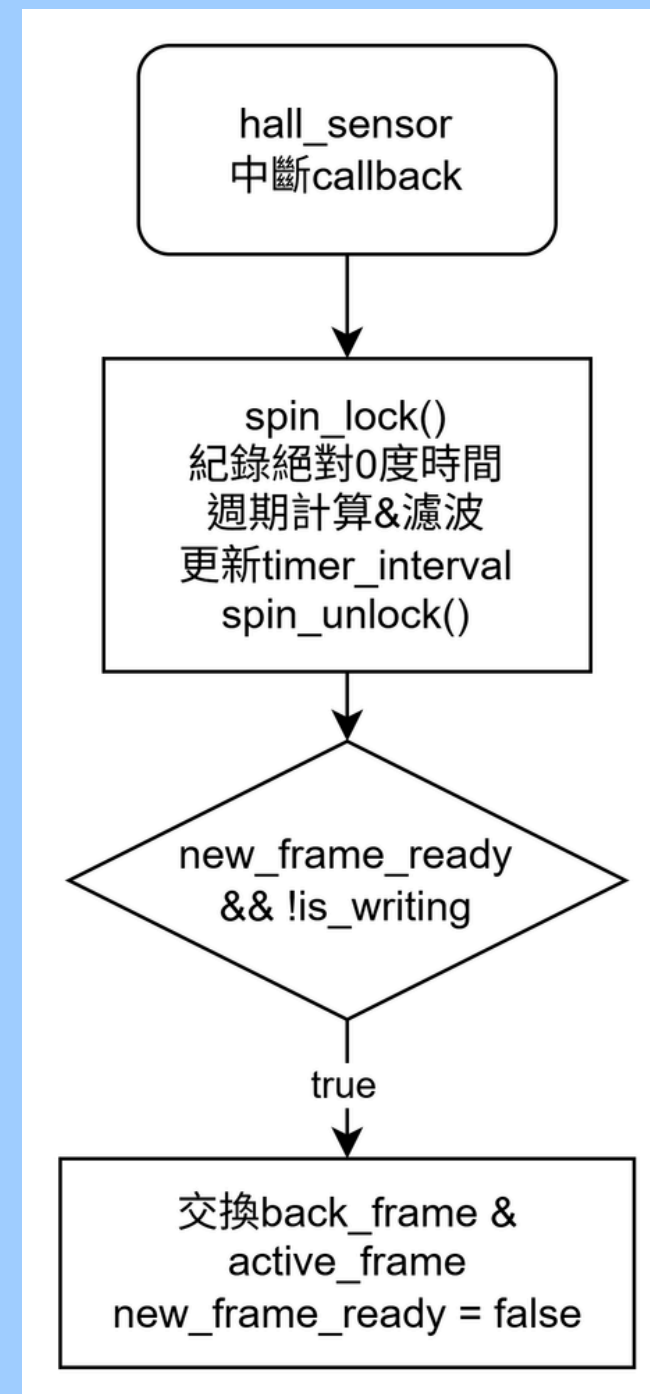
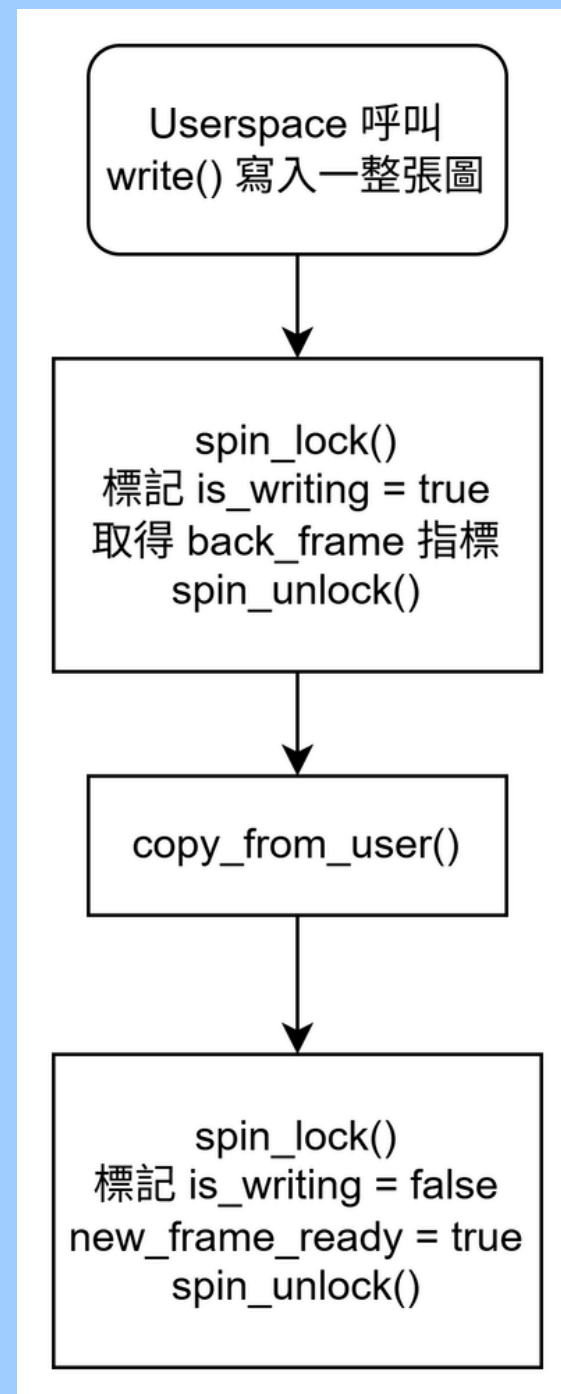


Magnet Driver : POV 系統的角度同步來源

- 接收 Hall Sensor 的 interrupt 與轉速判斷
- 記錄每次 Hall 觸發時間，計算並過濾不合理週期，保留穩定轉速
- 給 userspace 判斷是否正在旋轉，避免誤判
- 將觸發時間 callback 給 POV driver 進行同步



POV Display Driver : POV 顯示



POV Display Driver - Synchronization

Spinlock

- 保護 frame pointer、Hall time 與 current period 等共享資料
- POV 的畫面刷新需要即時性，加上 Linux kernel 禁止在硬體中斷中休眠。因此捨棄會導致執行緒睡眠的 Mutex，改用Spinlock。

Atomic

- 送出非同步 SPI 傳輸前，driver 會先檢查 SPI 是否仍在忙碌，避免 SPI 傳輸尚未完成時再次送出

Double Buffer

- userspace 寫入 back buffer，kernel 顯示 active buffer
- 新的完整 frame 寫入完成後，不會立刻取代目前畫面，而是在下一次 callback 時交換 buffer，避免畫面撕裂

POV Display Driver - Device Tree Overlay

- 為何不直接用 spidev ?
 - RPi 提供的 spidev 只能於 Userspace 使用，容易受Scheduler影響導致時間延遲
- 解法：
 - 撰寫自訂的 SPI Client Driver 與 .dts（編譯成 .dtbo）。讓 Linux 啟動並掃描 Device Tree 時，直接將SPI硬體節點綁定到 POV driver 中並與其他功能整合，建立完整的即時顯示流程

```
/ {
    compatible = "brcm,bcm2835";

    /* 第 1 步：關閉預設的 spidev0 */
    fragment@0 {
        target = <&spidev0>;
        __overlay__ {
            status = "disabled";
        };
    };

    /* 第 2 步：在 spi0 控制器上，掛載你的驅動 */
    fragment@1 {
        target = <&spi0>;
        __overlay__ {
            #address-cells = <1>;
            #size-cells = <0>;
            status = "okay";

            pov_ap102@0 {
                compatible = "custom,pov_ap102";
                reg = <0>; /* 解決 reg 警告：對應 @0 */
                spi-max-frequency = <15000000>; /* 15 MHz */
            };
        };
    };
};
```

Userspace Space

使用的 OS 機制

timer

- 以 60 Hz 的頻率更新狀態機以及渲染畫面

signal

- 接收硬體按鈕與 timer 事件通知

thread

- 實現 client-sever model
- Server thread 負責監聽，尚未收到連線會進入阻塞等待
- 有新的 controller client 連入，就建立獨立的 client handler thread 接收 controller 傳送的資料

Queue

- input queue 紀錄本機按鍵輸入與遠端手把輸入
- UI 與遊戲邏輯只需要從 queue 取得按鍵事件，不需要判斷輸入到底來自本機

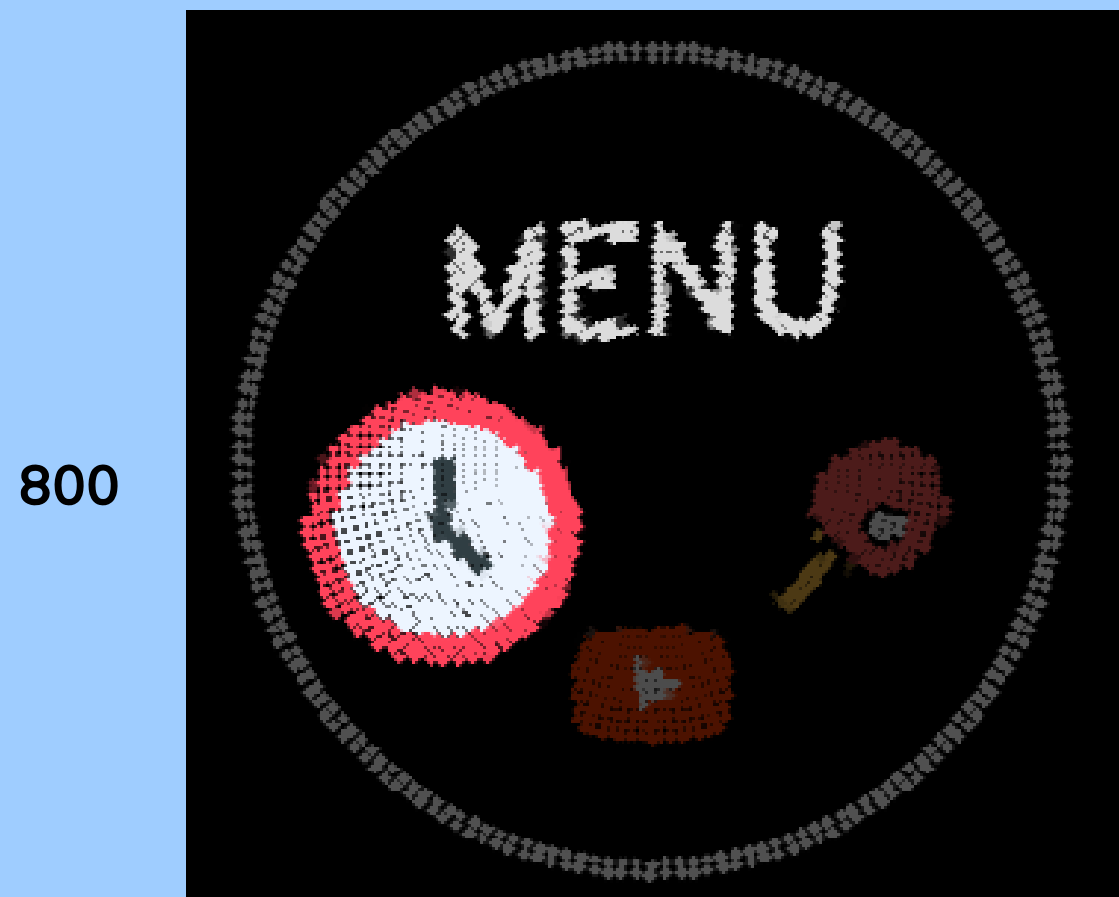
mutex

- 多個 thread 可能同時寫入共用 input queue，使用 mutex 保護 queue

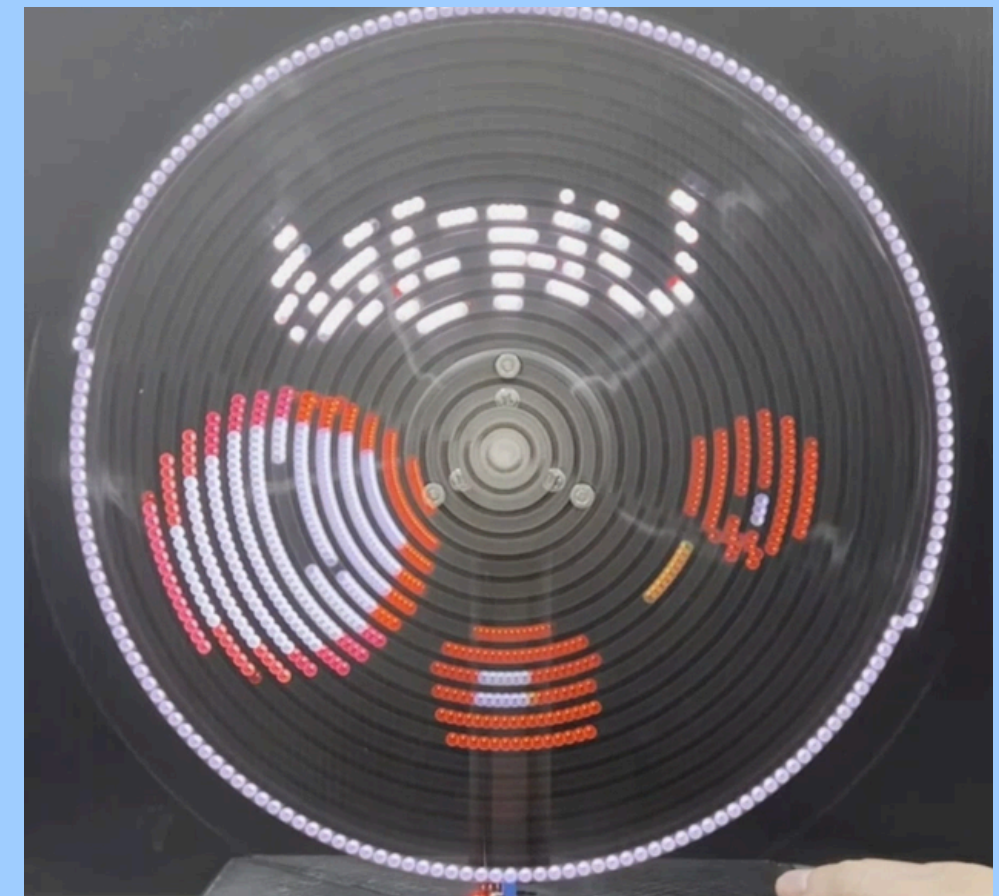
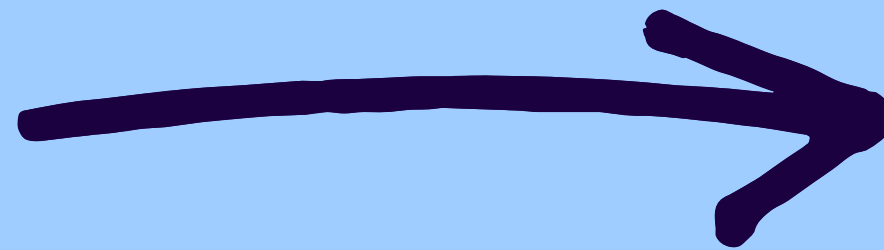
POVDisplay Class

將 2D 畫面轉 POV LED Frame

- 使用 OpenCV 產生 800×800 canvas
- 建立 Lookup Table，將 degree 與 LED index 對應到圖片座標
- 把 2D image 轉成 POV frame，並進行亮度與顏色補償
- 將 frame 寫入 POV display driver
→ 只負責產生完整 frame，即時顯示工作由 driver 負責。



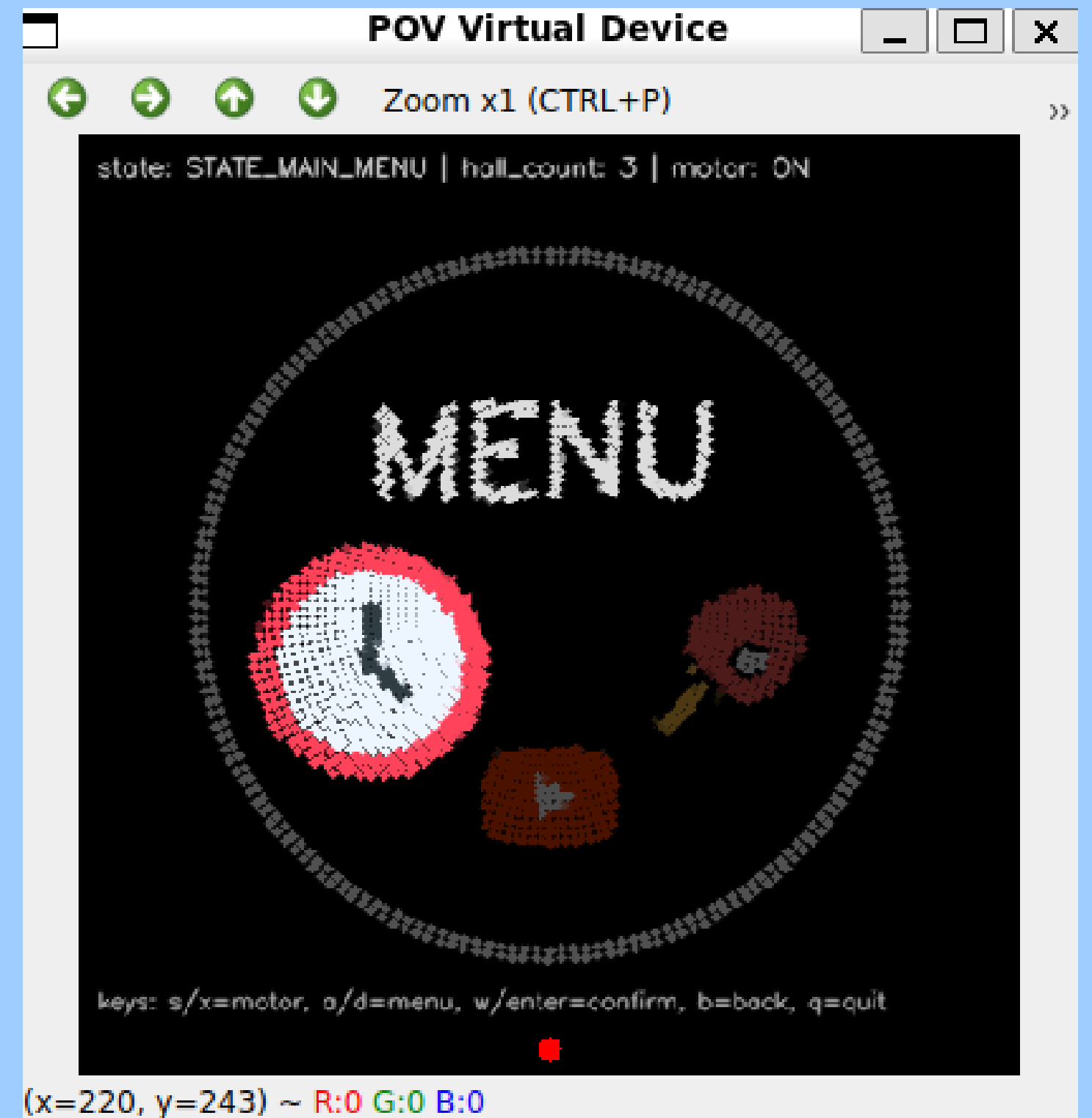
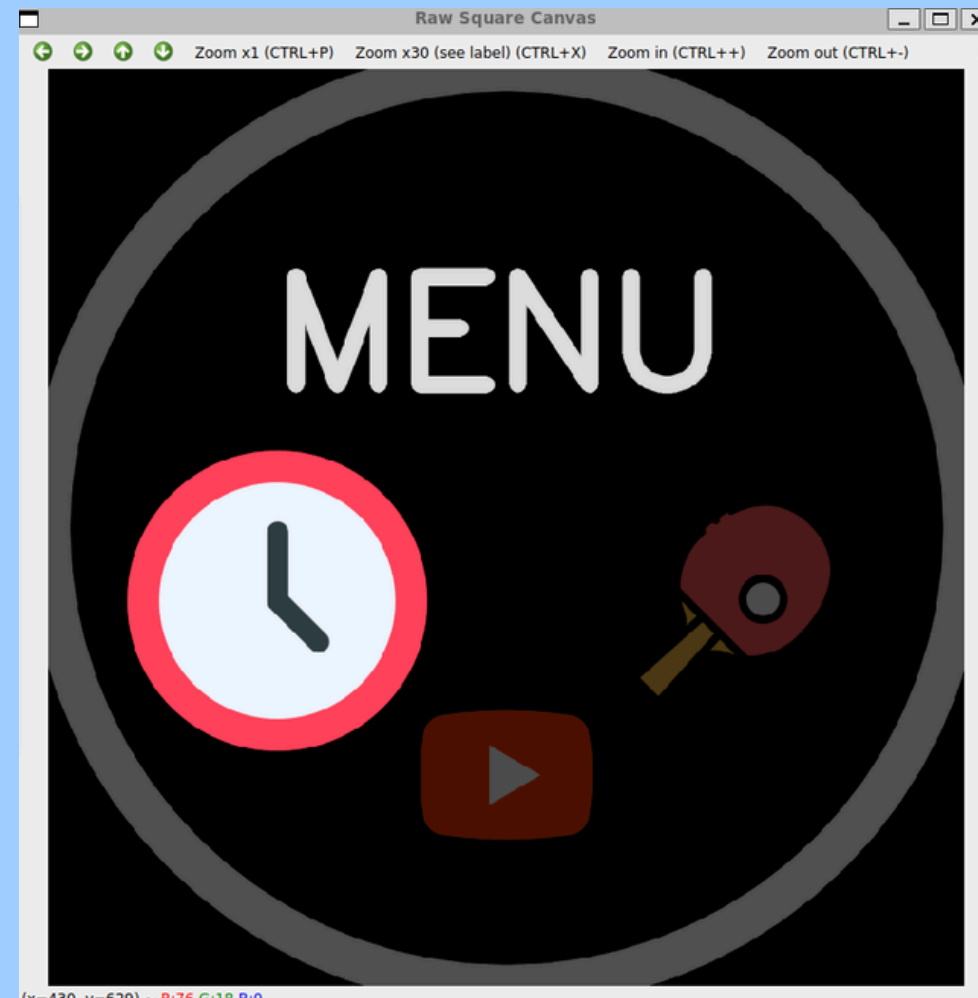
亮度與顏色補償



Virtual Device 模擬器

在沒有實體硬體時測試 UI 與狀態機

- 模擬 POV 顯示效果
- 模擬 Hall count 與馬達啟停
- 測試 menu、clock、video、game 等功能



Demo



遇到的問題與解決方法

- 問題 1：高速旋轉時畫面抖動

解法：Hall sensor 同步 + period 低通濾波 + time-based slice estimation

- 問題 2：userspace 更新畫面可能造成撕裂

解法：kernel double buffer + Hall callback 時交換 buffer

- 問題 3：SPI 傳輸不能被重複觸發

解法：atomic_t spi_busy 保護 SPI 傳輸狀態

- 問題 4：按鈕彈跳或 release 偵測不穩

解法：雙邊緣中斷 + delayed work debounce

- 問題 5：開機後 Wi-Fi 或 server 可能尚未就緒

解法：shell script 持續重試連線與 ping server

本專題使用到的 EOS 概念

- I/O Subsystem & Driver - 撰寫Linux Character Device Driver (`.ko` 模組)
- Interrupt (ISR) - 中斷服務函式，處理外部硬體（霍爾感測器、按鍵）的非同步事件
- Timer - 實現精準的定時callback，對齊馬達的轉速
- SPI - 分別控制燈條每顆燈珠顏色與亮暗
- Server/Client - 利用 Linux Socket API 建立多裝置連線
- thread - 用 thread 去處理 client 的請求
- atomic operation - 保護 client 端的操作，避免在操作中被打斷
- Mutex - 保護共享資源，避免在多個 client 同時連線時出現不可預期的操作
- Spinlock - 實作 button debounce
- Virtual device - 在實體裝置做出來之前，先用虛擬裝置進行程式的開發

Q & A